

New York University — Tandon School of Engineering
Department of Technology Management and Innovation
IE-GY 9113B — Systems Integration: From ERP to Agentic AI
Study Note — Week 1: Introduction — The Integration Backbone

Module 1: The Integration Foundation | Friday, September 4, 2026

Readings for this week	
Primary case	NYU Bobst Library / Jaggaer migration (2022), grounded in Ferguson (2025), “Migrating Collections Materials Purchasing to the Campus E-Procurement Platform,” NYU Libraries / ALA LRTS (June 2025). Open access: journals.ala.org/index.php/lrts/article/viewFile/8499/11823
Supporting case	Lidl / SAP “eLWIS” — the ~€500M program begun in 2011 and abandoned in 2018 after ~7 years. Reported by Computer Weekly: computerweekly.com/news/252446965/Lidl-dumps-500m-SAP-project
Go deeper (optional)	Ross & Weill, Enterprise Architecture as Strategy — Ch. 1 (the foundation for execution) and Ch. 2 (the four operating models). Westerman et al., Leading Digital — Introduction. These are reference, not required reading; dip in where you want the original voice.
Optional	Madison (2005) on process mapping — deeper detail behind the Four Lenses; not required, not held by NYU Libraries.
Further reading (optional)	On the leverage gap, if you want the current-state picture: BCG, “Are You Generating Value from AI? The Widening Gap” (Sept 2025) — bcg.com/publications/2025/are-you-generating-value-from-ai-the-widening-gap ; and Bain, “Unsticking Your AI Transformation” (2025) — bain.com/insights/unsticking-your-ai-transformation
Before next session	Reading this note before our first class is recommended, not required — the first session brings everyone up to speed. After class you’ll receive your team assignment, set up free accounts for the default-stack tools, and confirm your project company and its primary thread.

Start Here: How the Reading Works

A quick orientation, since this is your first study note. In this course the study note is the main text — not a companion to a textbook you read alongside it, but the primary reading you do before each week’s class. The books listed above are cited references to go deeper into when you want the original treatment, and each week a short real-world case is your applied reading; the note itself carries the teaching. Reading this Week 1 note before our first session is recommended but not required — the first class is built to bring everyone up to speed, whatever your background. From Week 2 onward you’ll receive each note a full week ahead, and the habit worth forming is to read it before class, so our time together can go further than the page.

The Backbone Is the Floor — Now Ask What It's For

This course moves through four stages — integrate, automate, make intelligent, make agentic — and everything you build sits on the first. Three business threads run through all of it, and you should learn to hear them as a recurring refrain: the Customer thread (taking an order and serving the person as one coherent promise), the Operations and Supply Chain thread (planning and synchronizing so the promise can be met), and the Cash Management thread (turning all that activity into cash quickly and reliably — cash flow being a leading reason businesses fail). Four company archetypes carry the lessons across scale — a global CPG manufacturer on SAP, a mid-market manufacturer on NetSuite, a Series-B startup stitched across Stripe, Shopify, and QuickBooks, and a fifty-person services firm living in HubSpot and Notion. Hold those in your head; almost every question in this course gets sharper when you ask it of all four at once.

First, the floor this week builds on — briefly, in case it is new to you. An enterprise system, or ERP (enterprise resource planning), is the software backbone a company runs its operations on, and its defining feature is a shared data model: one comprehensive database, with a single agreed definition of customers, products, accounts, and transactions, that every function reads from and writes to. Enter a customer order once, and finance, inventory, and shipping all work from the same fact. That shared foundation is the floor. One piece of history worth carrying with you: the expensive enterprise-system failures — FoxMeyer, Lidl, and many since — were rarely about bad software. The system reflects the organization; it rarely fixes it for free. This is the working minimum you need for Week 1; if you want a stronger foundation, the Week 0 primer develops it in full — the anatomy of an ERP, the shared data model, and the history of why these systems fail — and is available to anyone who wants the deeper introduction or a refresher.

Two questions organize the week, and the rest of the semester turns on them. First: what, precisely, is the integration backbone, and how do you read one? Here we deliver the full definition of the single source of truth, and we add the vocabulary — control points, decision points, the operating model, the two flows — you need to actually diagnose a company. Second, and harder: why does a backbone that works technically so often fail to deliver the value the business expected of it? The first question is architecture. The second is the seed of everything — the gap between a system that runs and a system that pays — and it is the question Week 2 exists to answer.

The backbone is table stakes, not the prize. Owning a working integration backbone is the price of entry to modern operations, not the source of advantage. This week defines the backbone and then turns to the harder question — why having one so rarely settles the matter of value.

One Fact, One Place: The Backbone as a Single Source of Truth

Strip the integration backbone down to its defining property and you are left with a single idea: a single source of truth. For any fact the business depends on — an order quantity, an invoice status, an inventory level, a credit limit — there is exactly one authoritative record that every function reads from and writes to. The structure that makes this possible is the shared data model

introduced above, one common definition of customers, products, accounts, and transactions, so that “customer 4471” means the same entity to finance and to fulfillment. The single source of truth is what that shared model is for. When it is absent, every department quietly keeps its own version of reality, and reconciliation — the endless labor of making the copies agree — becomes the organization's hidden full-time job.

A real backbone does more than store that one truth; it enforces the process that produces it. It will not let an invoice post against an order that was never approved, or ship goods that were never reserved. This is why we treat the backbone as both a single source of truth and a process enforcer — the two are inseparable, because the truth is trustworthy only if the steps that created it were followed in order.

Two end-to-end flows touch almost every integration decision you will make, so learn their names now and watch for them all term. Order-to-Cash, or O2C, runs from a customer's order through fulfillment to cash collected. Procure-to-Pay, or P2P, runs from a purchase requisition through receipt to a supplier paid. If you can trace these two flows through a company's systems — where a fact is born, where it is rekeyed, where it stalls — you can locate most of that company's integration strengths and most of its wounds. We will trace one of them, P2P, through a real case before this note is done.

To make those flows concrete rather than abstract, trace one. Take the mid-market manufacturer running order-to-cash on NetSuite. A sales rep enters a single order; if the backbone is doing its job, that one act reserves the inventory, schedules the shipment, generates the pick list, and stages the invoice — one entry, and every downstream function now knows what it needs to know. Now picture the same order in a company without a real backbone: the rep keys it into a CRM, someone re-keys it into a spreadsheet for the warehouse, a third person types the shipment into the accounting system, and a fourth reconciles the eventual invoice against a paper packing slip. Same order, four re-entries, four chances for the numbers to diverge — and every divergence is a customer who was promised something the warehouse never heard about, or an invoice for goods that shipped short. The flow is identical; the presence or absence of a single source of truth is the entire difference between the two companies.

This is why the backbone is non-negotiable at every scale, even though it looks completely different at each — and it is worth seeing it through our four archetypes, because you will meet all four in a career. For the global CPG manufacturer, the single source of truth is SAP: one enormous database that a sales-and-operations planning process reads from to balance demand against supply across dozens of plants. For the fifty-person services firm, it is a far humbler arrangement — HubSpot holds the client, an invoicing tool holds the money, a shared workspace holds the work — and the single source of truth is really a discipline about which tool owns which fact, enforced by people rather than software. For the Series-B startup, it is the integration layer stitching Stripe, Shopify, and QuickBooks so that a sale on the storefront becomes a charge and a ledger entry with no one re-typing it. Three radically different architectures; one identical requirement. Hold that in mind whenever someone tells you their company is too small for all this: they are not too small to need one authoritative version of their own facts — they simply have a smaller, cheaper way to get it.

One caution before we move on, because it is the most common misunderstanding of what a backbone is. Even companies that consolidate onto a single ERP rarely replace everything with it. The realistic picture is a hub with satellites: the ERP holds the core financial and operational truth, and specialized applications — a warehouse system, a CRM, an industry-specific tool the business cannot live without — orbit it and integrate into it. “One backbone” almost never means “one system”; it means one authoritative core that the specialists feed and draw from. Reading a backbone, then, is partly reading those seams — which is exactly where this week’s case goes wrong.

There is a prior choice hiding behind all of this, and it is worth naming because too many companies never consciously make it. Two distinctions matter. The first is the backbone’s type: it may be a single ERP, or a multi-SaaS stack of specialist tools stitched together (the startup’s Stripe, Shopify, and QuickBooks). Cutting across that is a second choice — whether the backbone is bought packaged, taken largely as the vendor ships it (SAP, Oracle, NetSuite as configured products), or custom-built, engineered to the company’s own specification. Packaged buys speed and upgradability at the cost of fit; custom-built buys fit at the cost of the maintenance burden we just watched sink Lidl. Which combination a company adopts is a strategic decision with long consequences — not a default to whatever a vendor happens to be selling, and not a question that must be answered the same way for every corner of the business. Where to standardize on the package, where to buy a specialist, and where to build — the discipline of concentrating effort where value is genuinely created or destroyed — is the harder question, and we take it up in full next week. For now, hold the two distinctions in view, and the fact that choosing among them is yours to make deliberately.

A single source of truth that also enforces the process. The backbone’s job is to hold one authoritative record of every operational fact and to make the steps happen in the right order. Order-to-Cash and Procure-to-Pay are the two flows that run through nearly every integration; learn to read them.

Where Data Is Born: Control Points

Trace either flow and you notice the line is not smooth — it has a few moments that matter far more than the rest. A purchase order is issued. A serial or lot number is generated. Credit is approved. An invoice is posted. These are control points: the instants where the business actually commits to something and a new, authoritative fact comes into existence. The backbone lives or dies at these points, because a control point is where data is born — and therefore where governance attaches (who is allowed to commit this?), where an error is cheapest to catch, and, later in this course, where automation and AI agents will plug in. Learn to find them: in Lab 1 you will mark them on your own process map, which is what the star denotes below.

ORDER-TO-CASH (O2C)

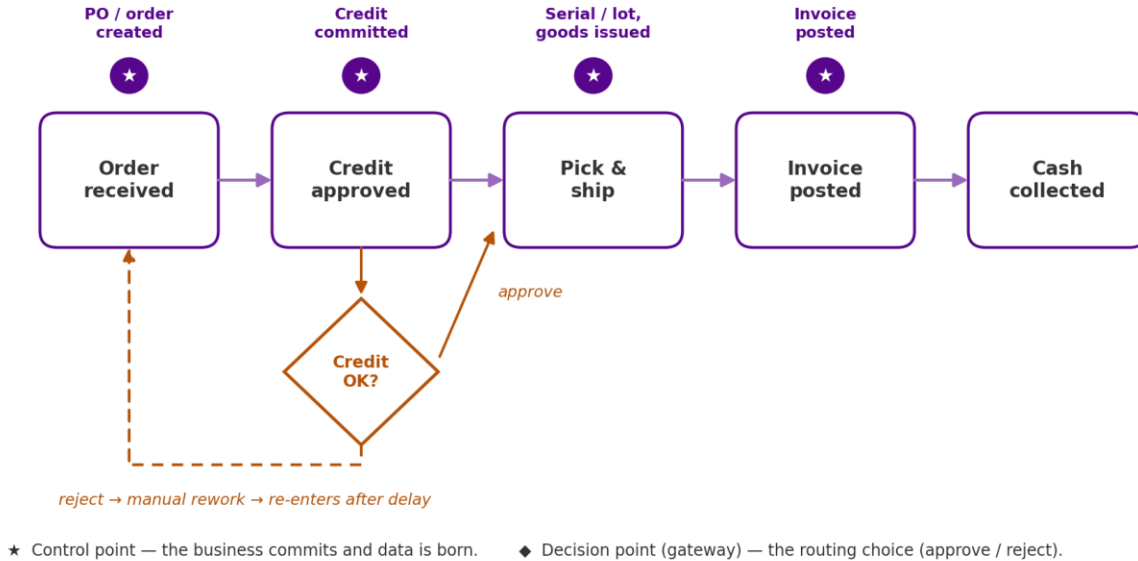


Figure 1. An Order-to-Cash flow: control points (stars, where data is born) and a decision-point gateway (diamond) with its rework loop (where the path is chosen). A simplified flow, not strict BPMN. Figure, IE-GY 9113B.

This is also why, all term, we read the process before the data. A control point is a step in a process; the record it creates exists only because the work reached that step. Data hangs on a process, not the other way round — so when you diagnose a company, you map the business process and its control points first, and you fit the data to it last. Business first, data last. You will feel this directly in Lab 1, where you map and pressure-test the process before generating a single synthetic row.

Control points are where the business commits and data is born. Issuing a PO, generating a serial number, approving credit, posting an invoice — these are where governance attaches and where automation will later plug in, so they are the first thing you mark when you map a process. And because data is created by a process reaching a control point, you read the process first and the data last.

Where the Path Forks: Decision Points

Control points tell you where data is born; decision points tell you where the flow chooses a path. They are genuinely different, and conflating them is a common analytical mistake. A control point is where the business commits and a record is created — a PO issued, a serial number generated, an invoice posted, credit approved — which is where governance and, later, automation attach. A decision point is where a routing choice is made: approve or reject, send the work down one path or another. Every process is a chain of both, and you cannot read one without reading the other.

One decision pattern is worth singling out now, because it is where value quietly leaks. When a decision point rejects the work in front of it — a mismatched invoice, an order that fails a credit

check, a form filled in wrong — that work does not simply stop. It is pushed back, handled manually to bring it into line, and re-enters the flow only after significant rework. Each loop adds delay, and each manual touch adds a fresh chance for error, so rejections are where cost and error compound fastest. In Figure 1, the diamond is that decision point and the dashed loop is the rework it triggers. Learn to spot these loops: they are the richest targets in Lab 1, and they are exactly where the course's later question — which steps a rule-based robot can handle and which need genuine judgment — gets decided.

Control points and decision points are not the same. A control point is where the business commits and data is born; a decision point is where the flow is routed — approve or reject, one path or another. The expensive pattern is rejection: work sent back, reworked by hand, and re-entered only after delay, compounding cost and error. Both get marked when you map a process.

A First Look at Friction: The Four Lenses

If control points are where data is born, friction is where value leaks — and you need a disciplined way to see it, not just a hunch that something is slow. The course borrows a simple, durable diagnostic from process-improvement practice (Madison, 2005, the optional reference above): four lenses you hold up to any step to ask where it is hurting. We only preview them this week so they are not cold when you reach the lab; the full diagnostic treatment comes next week, and you will apply them first in the Week 2 pre-work and then in the Week 3 lab.

Lens	What you look for at a step
Frustration	Where the work irritates the people doing it or the customer receiving it — usually a tell for hidden rework, handoffs, and workarounds.
Time	Where the process waits, queues, or doubles back — elapsed time far exceeding the actual work time.
Cost	Where effort, rework, or error-correction consumes money out of proportion to the value the step adds.
Quality	Where errors, exceptions, and defects are produced — and then have to be caught and fixed somewhere downstream.

Notice that a lens looks at a process, not at a database — the same point again, from the other side. You find the leak in the work, then ask what data would prove it. Keep the four in your pocket; next week they become the spine of how we diagnose a failure before redesigning it.

The Four Lenses are a preview. Frustration, Time, Cost, Quality (Madison, 2005) — four lenses for seeing where a process leaks value. Previewed now so they are not cold in the lab; taught fully in Week 2; applied in the Week 2 pre-work and the Week 3 lab.

Two Questions Decide Everything: What Is the Same, and What Is Known

Reading a backbone is not only about tracing flows and marking control points; it is about understanding the organizing choice the company made before it bought any technology. Ross and Weill's central move — developed in the chapters listed under “go deeper” this week — is to insist that a firm answers two prior questions about how it intends to operate, and that these are two separate decisions executives too often blur into one.

The first is standardization: how identically should a process run across the company, regardless of who performs it or where? Standardizing a process lets you measure, compare, and improve it, and it drives variability out, which lifts throughput and efficiency — but it has a price. It limits local innovation, and the transition usually means ripping out systems that worked perfectly well, which is expensive and politically bruising. The second is integration: how much should units share data — between processes, so a transaction flows end to end, or across processes, so the company shows a single face to the customer? Integration buys coordination, transparency, and speed, and its hardest part is almost always the data itself. Units must agree on common definitions, even for a word as innocent as “sale,” which one unit dates from the signed contract, another from the cash received, and a third from the product delivered. Until they agree, there is no single source of truth to share.

These are not abstractions; they are two questions you can put to any company, and they are exactly how you will place your own project company later this term. Ross and Weill phrase them precisely. First: how much does the successful completion of one unit's work depend on the availability, accuracy, and timeliness of another unit's data? That answer sets your integration requirement. Second: how much does the company gain from having its units run their operations the same way? That answer sets your standardization requirement. Two questions, two answers, and you have located a company on the map.

Standardization and integration are two separate choices. Before any technology, a company decides how identically its processes should run (standardization) and how much its units should share data (integration). Ross and Weill's two placement questions — how much one unit depends on another's data, and how much the firm gains from units operating alike — are exactly how you will classify your own company.

Four Stances, and Four Real Companies That Took Them

Cross the two questions and you get four operating models — four deliberate stances a company can take toward how it runs. Ross and Weill anchor each in a real firm, and those companies make the abstraction concrete far better than definitions do.

Stance	Standardization × integration, the company that shows it, and what it buys
Diversification	Low / low. JM Family Enterprises runs several automotive businesses — Toyota distribution, auto finance, an F&I group, the world's largest Lexus dealership — that feed one another but share little beyond a parent and a few central services. Units are autonomous; the company grows through their individual success and through acquisition.
Coordination	Low standardization / high integration. Merrill Lynch's Global Private Client lets thousands of advisers across hundreds of offices each own their client relationships their own way, while all of them draw on one shared platform of product and customer data. Integrate the data, not the process.
Replication	High standardization / low integration. McDonald's is the reference point — a proven formula stamped identically into every unit, each running its own data. TD Banknorth grew roughly tenfold by acquiring community banks and dropping each onto one standardized foundation while letting it run independently. The road to economies of scale.
Unification	High / high. Dow Chemical runs its global chemicals business on one ERP with centrally mandated databases and roughly sixty percent of its work processes standardized worldwide — one process and one truth everywhere. Unification suits commodity-like operations where driving out variation pays.

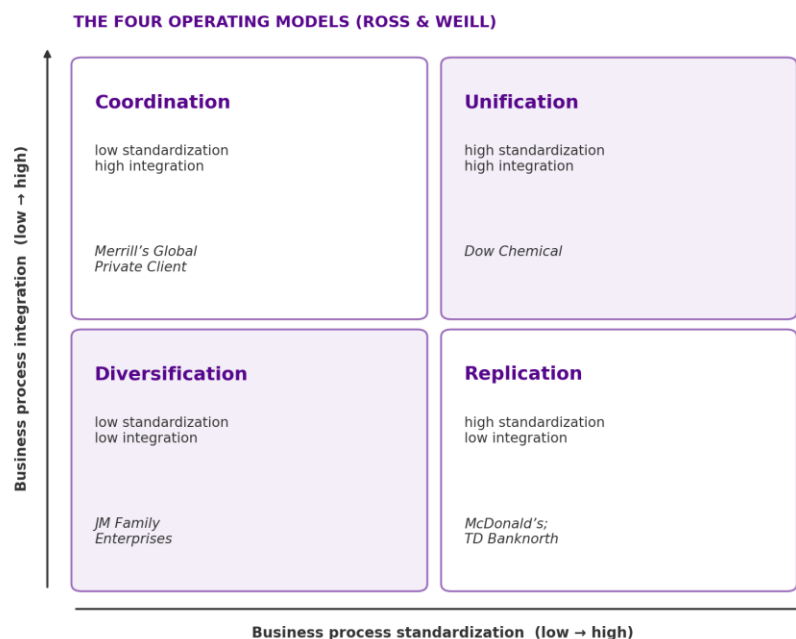


Figure 2. The four operating models on the two dimensions — adapted from Ross & Weill (2006). IE-GY 9113B.

There is no best quadrant — only a stance that fits your strategy or fights it. A diversified holding company gains nothing by forcing one process onto unrelated businesses; a global discounter cannot afford four different ways to run a store. And the choice taxes someone every time: standardization takes autonomy from local managers, and integration takes ownership of “their” data away from units that were content to keep it. The deepest point Ross and Weill make is that once you choose a stance, it stops being a description and becomes a driver — it starts to shape

which strategies the company can pursue cheaply and which it cannot. (Large companies need not pick just one: Johnson & Johnson runs Diversification overall, yet its U.S. pharmaceutical group runs Coordination and its European Janssen unit runs Replication.) Classifying your project company in one of these quadrants is exactly Section 3 of your board memo, the Integration Backbone Assessment — and if you want the full original treatment, Ross develops all four models in Chapter 2.

One more thing the map tells you, and it is the reason the choice matters for strategy rather than only for IT: each stance positions a company for a different kind of growth. A Coordination firm like Merrill grows by selling more to customers it already understands deeply — the integrated data makes the next product cheap to offer. A Replication firm like McDonald's grows by stamping its proven formula into one more location at a known, low cost. A Unification firm like Dow grows by leveraging its standardized scale into new markets, and a Diversification firm grows by acquiring and letting the new unit run. So when you classify your project company, you are not filing it into a box — you are predicting which moves its backbone will make cheap and which it will make expensive. That prediction is precisely the argument your board memo has to make, which is why this classification is worth getting right.

To see why this is not academic, take a company Ross and Weill follow through the change. A European packaging firm had grown up as a set of country-based units, each with its own ERP, its own order process, even its own pricing — so vividly fragmented that corporate customers learned to take the same order to several countries at once and play the units against one another on price. Management decided its core operations would run better under Unification and replaced the country systems with one central ERP reached through a browser, on one product list and one price list. The payoff was concrete: adding a product with a new pricing structure had meant updating fifteen separate systems over weeks; now it was a single change made in hours. But notice the cost — country managers who had set their own products, pricing, and promotions lost much of that authority, and they resisted. That is the standardization tax made flesh, and it is a preview of Week 2, where we watch what happens when the human side of such a change is ignored.

Four stances, each embodied by a real firm. Diversification, Coordination, Replication, Unification — the four operating models sort companies by how much they standardize and how much they integrate, and each buys a different kind of growth. Once chosen, the stance drives strategy rather than merely following it. Classifying your company here is the core of Board Memo Section 3.

The Foundation That Frees You — and How to Spot Its Absence

Here is the paradox at the heart of Ross and Weill's argument, and it is worth sitting with because it explains the entire course. Digitizing your core processes makes each individual process less flexible — it is now embedded in software, not improvised — and yet it makes the whole company more agile. Think of an athlete: the muscles and reflexes are not easily changed, and that very fixedness is what lets the athlete react, improvise, and invent in the moment. A firm that has

digitized what does not change is free to focus on what does. The foundation for execution becomes a platform for innovation precisely by making the routine boring.

The numbers in the reading are worth carrying into your project conversations. In Ross and Weill's survey, the roughly one-third of companies that had digitized their core processes enjoyed higher profitability, faster time to market, better access to shared customer data, lower risk of mission-critical failures, and — strikingly — about a quarter lower IT costs. The foundation is not a luxury that costs more; done right, it costs less.

So how do you know a company lacks one? Ross and Weill offer a checklist of symptoms, and you should learn to hear them in the wild — in a case, in an interview, in your own employer. Different parts of the company give different answers to the same customer question. Meeting a new regulation is a massive, top-driven scramble. Every new initiative feels like starting from scratch. IT is perpetually the bottleneck. The same activity runs through several different systems in different corners of the company. And — mark this one, because we are about to watch it happen — a significant part of people's jobs consists of taking data out of one system, massaging it, and entering it into another. That last symptom is not a footnote; it is the precise shape of the case we turn to shortly, and the reason a technically successful project can still leave an organization carrying real, daily cost.

The foundation that frees. Digitizing the core makes each process less flexible but the company more agile — the routine becomes reliable, freeing attention for what changes. The absence of a foundation has tell-tale symptoms; the sharpest is people spending their days carrying data from one system into another. Watch for it — it is the bridge to this week's cases.

Why a Backbone That Works Still Fails to Pay

Here is the question the whole course turns on, and it is worth sitting with. A company can select an integration backbone, implement it, survive the go-live, and end up with a system that genuinely works — data flows, transactions post, the screens do what they should — and still find that the business is no better off. The promised savings do not materialize; the customer experience is unchanged; the cash still moves at the old speed. The distance between a backbone that is deployed and a backbone that delivers value is what this course calls the leverage gap. Closing it is the actual work of the next thirteen weeks; the technology is only the ticket that lets you start.

And this is not a rare pathology. Gartner projects that by 2027, more than seventy percent of recently implemented ERP initiatives will fail to fully meet their original business goals — and that as many as a quarter of those will fail catastrophically. Read the number carefully: it is not saying the software does not run. It is saying that most of the time, even a functioning system does not deliver what the business set out to get. That is the leverage gap, measured.

Your Week 1 reference reading makes the point from the outside. Westerman and his colleagues studied more than four hundred large, traditional firms — pointedly excluding the Silicon Valley

stars and the startups — to learn what technology does in the ninety-plus percent of the economy that does not do technology for a living. Their finding is the one to carry: the firms that win are not the ones that spend the most. Winning takes two capabilities at once — the digital capability to work differently, and the leadership capability to set a vision and drive the change through a large organization. Firms that pair both, which they call Digital Masters, are rare, and they run about twenty-six percent more profitable than their industry peers. Buying the technology is not the thing. Leading with it is.

It is worth being precise about those two capabilities, because the distinction is the whole lesson. Digital capability is investment in the technology itself — the systems, the data, the automation on top of the backbone. Leadership capability is the far scarcer thing: a clear vision of how the business should actually be different, and the governance and sheer persistence to drive that change through an organization that would rather not change. Firms rich in the first and poor in the second buy impressive tools that never alter how the company really works — they end up with the most technology and the least to show for it. The backbone is digital capability made concrete. The leverage gap is what opens when digital capability arrives without the leadership to wield it, which is why the rest of this course spends as much time on adoption, governance, and process as it does on the tools themselves.

The gap also shows up differently on each of our three threads, and learning to see it there is part of the diagnostic you are building. On the Customer thread, a backbone that works technically can still leave a customer facing a company that does not seem to know its own promises — an order status no one can answer, a return that falls between two systems. On the Operations and Supply Chain thread, the gap looks like a plan that is technically integrated but that no one trusts enough to act on, so the old spreadsheets quietly persist beside the new system. On the Cash Management thread — where this week's case sits — it looks like a payment process that is faster on paper yet still leaks work-weeks into reconciliation. One backbone, three faces of the same gap.

That is the leverage gap stated as a research finding: the asset is not the software but the organizing intelligence wrapped around it. Every later module in this course is an attempt to close that gap on a specific frontier — automate the routine, make the automation intelligent, make the intelligence agentic — but each of them only pays if you have first understood why a working backbone, left alone, so often does not. Next week we put that failure under a microscope.

The leverage gap is the real subject. A backbone that works technically can still fail to move the business; the gap between deployment and value is what the course exists to close. Across 400-plus traditional firms, the winners were not the biggest spenders — they paired digital capability with the leadership to drive it. The asset is the organizing choice, not the technology.

Reading a Backbone in Our Own Backyard: NYU Bobst Library

You do not have to look to a Fortune 500 to see these forces; they are operating not too far from this classroom, in NYU's own Bobst Library, and the episode is documented in the open-access

article you are reading this week — Ferguson (2025). It is a small case by design: the point is not scale but clarity, and a procure-to-pay flow you can hold in your hand.

The problem. From 2014 to 2021, Bobst paid most of its collections invoices through a homegrown “invoice export workflow” that bypassed iBuy, the university's standard procurement platform. Staff ordered and received inside the library's own system, and a weekly job pushed invoice data straight to Accounts Payable. To the people using it, it felt efficient. To everyone outside the library, it was opaque — the exported data carried no record of who had authorized the spending, no metadata about what was purchased, and no copies of the invoices themselves.

What happened. An early-2021 internal audit found exactly that opacity and recommended adopting the standard platform. Bobst migrated and went live on iBuy on January 1, 2022. The opacity dissolved: collections spending became visible in the university's central systems like any other expenditure, every order carried its approver, and invoices lived centrally.

The twist. Solving one integration gap opened another. iBuy and the library's own system are not integrated with each other, so staff now enter each purchase twice — once for the library's inventory and reporting, once for the university's procurement and payment. The library measured the cost: roughly ten additional work-weeks in a single year, purely from the new double handling.

The library did try to close the new gap. It built an Airtable workflow to bridge the two systems automatically — but the automated emails it generated were swallowed by the university's spam filters, and the bridge never reliably held. Layered on top of all this, the library was simultaneously migrating its own catalog system from Aleph to Alma, so the very platform on one side of the seam was moving while the seam was being stitched. None of this is exotic; it is exactly the texture of real integration work, where the ground shifts under you and the elegant fix meets a spam filter. What makes the episode teachable is that the university could actually measure the residue — those ten work-weeks — a number most organizations never bother to compute and therefore never see.

The lesson. Notice what the backbone did right — it restored a single source of truth for central finance, pulled a rogue process into the standard flow, and put an authorizer and a retained invoice back on the record. And yet it re-created the very symptom of a missing foundation, people spending their days moving data between systems, at a new seam. That is Week 1 in miniature: integration is not a state you arrive at; it is a gap you keep closing, and closing one can open the next. On our three threads, Bobst sits mostly on Cash Management, with an Operations overlay — the diagnosis Lab 1 will ask you to perform on a company of your own.

Map Bobst onto the four stances and the friction becomes legible. The university centrally wants shared financial truth (high integration) and is pushing everyone onto one procurement platform (rising standardization) — it is reaching, in finance at least, toward Unification — while the library, with its homegrown workflow, behaved like an autonomous Diversification island. The whole episode is the friction of pulling a Diversification-minded unit into a Unification-minded enterprise. It also restored two controls worth naming: the three-way match, where a payment is released only against an approved order and a confirmed receipt, and the encumbrance, funds committed against an order the moment it is placed so the money cannot be spent twice. And lest this read as a peculiarly academic problem, Ross's own research carries the government of Washington,

D.C., which went “from worst to first” by building a foundation across dozens of agencies — the logic applies identically to companies, governments, and non-profits. Bobst is that same lesson at library scale.

Bobst, read closely. Adopting the standard backbone solved a real opacity problem and, because two systems still don't talk, re-created the textbook symptom of double data entry at a new seam. A technically successful migration can still impose daily cost when integration is only partial.

The Same Lesson at Enterprise Scale: Lidl and SAP

If Bobst is the leverage gap at library scale, Lidl is the same lesson at nine figures. Beginning in 2011, the German grocery giant set out to replace its aging inventory system with a custom SAP platform it called eLWIS. Seven years and a reported €500 million later, in 2018, Lidl abandoned the project and reverted to the legacy system it had set out to retire — one of the most expensive enterprise-software failures on record, reported by Computer Weekly and Handelsblatt among others.

The reason is a gift for this course, because it is not a story about bad software. Lidl runs its business on a lean, low-margin discipline in which inventory is valued at the price the company paid for goods. Standard SAP for Retail, by contrast, is built around the retail price. Faced with that mismatch, Lidl chose to bend the software to its existing processes rather than bend its processes to the software — and the customization compounded until the program collapsed under its own weight. The backbone was, in a narrow sense, deliverable; it was never adoptable, because the operating logic underneath it was never reconciled with the system on top. That is the leverage gap written large: a company can spend half a billion euros on a backbone and still get no value from it, because value never lived in the software. In the language of the four stances, Lidl tried to run a Unification-style standard system on top of an operation whose logic it refused to standardize — the operating-model tension of the previous section, played out at half a billion euros.

There is a name for Lidl's mistake, and it is worth knowing because it is the single most repeated error in enterprise systems: choosing to customize rather than to configure. A configured system stays close to the vendor's standard, so it survives upgrades and patches; a heavily customized one drifts further from that standard with every change, until routine updates break it and the cost of keeping it alive outruns its value. Gartner's own post-mortems put over-customization near the center of ERP failure — scope customized to today's requirements cannot keep up with tomorrow's, and the system eventually has to be replaced. Lidl is that pattern at its most expensive.

Bobst and Lidl — the same gap, two scales. A partial integration that imposes daily cost, and a half-billion-euro system abandoned after seven years, are the same lesson at different magnitudes: a backbone delivers value only when the organization and its processes are reconciled with it, not when the software merely runs.

How to Read a Backbone: The Method You Will Use All Term

Everything this week has given you is really a single diagnostic — a repeatable way to walk into any company and read its integration backbone, which is exactly what Lab 1 and your capstone will ask you to do. It is worth assembling the steps in one place, because you will run them again and again, on the cases we study and on the company you choose.

1. Trace the flows. Pick the Order-to-Cash or Procure-to-Pay flow that matters most to the business and follow it end to end. Where is each fact born, where is it rekeyed by a human, where does it stall waiting on another system? The re-entries and the stalls are your first map of the wounds.

2. Mark the control points and decision points. On that flow, mark where the business commits and data is born, and — distinctly — where the flow is routed. Pay special attention to the reject-and-rework loops at decision points; they are where value leaks fastest and where the later rule-versus-judgment call will be made.

3. Classify the operating model. Ask Ross and Weill's two questions — how much does one unit depend on another's data, and how much does the firm gain from units operating alike — and place the company in one of the four stances. This tells you what the backbone should be optimizing for, and whether its current shape fits or fights the strategy.

4. Run the Four Lenses. At the control points and the stalls, hold up Frustration, Time, Cost, and Quality. Where the work irritates the people doing it, waits in a queue, costs more than the value it adds, or produces errors someone else must fix — that is where value is leaking.

5. Listen for the warning signs. Different parts of the company giving different answers to the same question; every initiative starting from scratch; IT as the perpetual bottleneck; people spending their days moving data from one system into another. Each is evidence that the foundation is weak, whatever the software cost.

6. Name the leverage gap. Finally, ask the question that ties it together: does this company have a backbone that works but does not pay? If so, where exactly does the value fail to appear — on the Customer thread, in Operations, in Cash — and what would it take to close the gap? That answer is the seed of your recommendation.

Run those six steps and you have done, in miniature, what a transformation diagnostic does at a Fortune 500 — and what your board memo will argue at length. Bobst and Lidl are simply this method applied to two real companies at two very different scales; the sections above walked the steps for you so that, in Lab 1, you can walk them yourself. When you pick your company this week, you are choosing the backbone you will read with exactly these tools.

And read it early. The decisions that most determine whether a backbone ever closes its leverage gap — which system, which partner, which processes to standardize, which requirements are genuinely non-negotiable — are made at the very start, when they are cheapest to get right and, paradoxically, hardest to see clearly. As a project grows, those early choices calcify, and changing them later means costly rework. That is why enterprise transformation demands a different

mindset from a conventional IT project: an IT project asks “can we build it?”; a transformation asks “should the business work this way at all?” — and answers before the building starts. It is why this diagnostic comes first.

The method is the point. Trace the flows, mark the control points, classify the operating model, run the Four Lenses, listen for the warning signs, and name the leverage gap. Six steps that turn this week’s concepts into a repeatable diagnostic — the one you run in Lab 1, on your project company, and in every section of the board memo.

Where This Week Sits — and What It Hands to Week 2

The labs. Week 1 is the on-ramp to Lab 1 in Week 3, where you map one end-to-end process for your project company, mark its control points and decision points, and label three things on it: the worst friction (named by one of the Four Lenses), the best candidate for rule-based automation, and the one step that genuinely needs human judgment. Once your team is assigned this week, you’ll set up the Default stack (Airtable or Google Sheets with Gemini), which is guaranteed for the build, with the AWS and custom paths available by approval. Treat Bobst as a worked example of exactly the diagnosis the lab asks of you.

The capstone. You receive your team assignment after this first class, and once teams form you confirm your company and the primary thread its integration challenge most affects. That choice, plus a first backbone-context note — the stack in place, the key processes, the archetype it most resembles, and the primary thread — is Section 2 of your board memo, the Context and Background on which every later recommendation rests. You also receive the capstone storyboard this week and map your team’s final board deck to it, so each week’s homework fills a defined piece of the final presentation. Work with a real organisation you can learn about — a current or former employer, a family business, an internship host, or one a teammate has a genuine connection to; all the project data you generate is synthetic, so no confidential company information is ever used or required.

The hand-off to Week 2. We end on the leverage gap because Week 2 dissects it. There you will meet the delivery ecosystem and the legacy, multi-system reality that make real backbones so hard to change; Hammer’s argument to redesign the process before automating it (“obliterate, then automate”); the Four Lenses in full; and two cases a generation apart — Nike and Birmingham — that fell into the identical trap. Your Lab 1 pre-work is assigned there: pick your company, research its real friction through the Four Lenses, and draft a short research brief that gates the Week 3 build.

Your Study Questions

Bring rough answers to our first session; they are designed to be argued, not recited.

1. How does an integration backbone function as a single source of truth, and why is it non-negotiable regardless of company size? Anchor your answer in the shared data model,

and show that the failure mode is identical at every scale — from SAP to three stitched SaaS tools, the absence of one authoritative record turns reconciliation into a permanent tax. Use two of the four archetypes to make the point.

2. Bobst solved one integration gap and opened another. What does that tell you about integration as a goal? Read it as evidence that integration is a gap you keep closing rather than a finish line, and connect it to the leverage gap — a technically successful change that still leaves real cost on the table.

3. Lidl spent ~€500M and got nothing. Where, exactly, did the value fail to appear? Locate the failure in the mismatch between the company's operating logic and the standardized software — not in the technology itself — and say what a different sequence of decisions might have changed.

4. Where does your potential project company sit on the four-stance map, and which thread does its integration challenge most affect? Use Ross and Weill's two placement questions — how much one unit depends on another's data (integration), and how much the firm gains from units operating alike (standardization) — to place the company in one of the four operating models. Then name whether its integration challenge lands mainly on Customer, Operations and Supply Chain, or Cash Management. This placement is the core of Board Memo Section 3 and your bridge into the capstone.

5. Bring one example, from your own work or an industry you know, where integration is clearly working or clearly failing. A concrete, real instance you can reason about beats a tidy abstraction — and it is the raw material for your project choice.

6. Take a process you know and find one control point and one decision point in it. For the decision point, estimate the cost of a rejection — the delay and the manual rework a sent-back item triggers before it re-enters the flow. This is the muscle Lab 1 will exercise: seeing where a process commits, where it chooses, and what the choosing costs when it goes the wrong way.

Running Glossary

Integration backbone / ERP. The shared technology and data layer that holds a company's operational truth — whether a single ERP such as SAP or NetSuite, or a coordinated set of SaaS tools stitched together.

Single source of truth. Exactly one authoritative record for each operational fact, that every function reads from and writes to. The defining property of a real backbone.

Shared data model. One common definition of customers, products, accounts, and transactions, so a given entity means the same thing to every function.

Order-to-Cash (O2C) / Procure-to-Pay (P2P). The two end-to-end flows every integration touches: order through fulfillment to cash collected, and requisition through receipt to supplier paid.

Control point. A moment where the business commits and a new authoritative fact is created — a PO issued, a serial number generated, credit approved, an invoice posted. Where governance and, later, automation attach.

Decision point. A step where the flow is routed — approve or reject, one path or another — as distinct from a control point where data is born. The costly case is rejection, which sends work back for manual rework before it re-enters the flow.

Packaged / multi-SaaS / custom backbone. The three broad shapes a backbone can take — a bought packaged ERP, a stitched stack of specialist SaaS tools, or custom-built components — among which the choice is a deliberate strategic decision.

Standardization / integration. The two dimensions of an operating model: how identically processes run everywhere, and how much units share data. Together they define which of four stances a company occupies.

The four operating models (stances). Diversification, Coordination, Replication, Unification — the four combinations of low/high standardization and integration (Ross & Weill). How you classify a company's backbone for Board Memo Section 3.

Three-way match / encumbrance. Two procurement controls: releasing payment only against an approved order plus a confirmed receipt, and committing funds against an order the moment it is placed so they cannot be spent twice.

Leverage gap. The distance between a backbone that is deployed and one that actually delivers value. The central problem the course sets out to close.

The Four Lenses. Frustration, Time, Cost, Quality — four lenses for diagnosing where a process leaks value (Madison, 2005). Previewed in Week 1, taught in full in Week 2.